

AI / ML Engineer Interview Prep

50 Interview Questions - Fundamentals to Advanced | 5-Year Experience Level

Focus	What to prepare
Top priority	ML fundamentals, metrics, imbalance, model selection, overfitting, production debugging
Deep learning	Neural nets, activations, regularization, CNN/RNN/Transformers, attention
MLOps	Deployment, monitoring, drift, reproducibility, CI/CD, feature stores
GenAI	RAG, embeddings, evaluation, hallucinations, agents, security
Interview style	Answer concept -> example -> tradeoff -> production/business impact

How to use this guide: First answer each question aloud without looking at the answer points. Then compare your response, fill the gaps, and repeat until you can answer clearly in 60-120 seconds.

Section 1 - ML Fundamentals

1. What is the difference between supervised, unsupervised, and reinforcement learning?

What a strong answer should cover: Supervised learning uses labeled data; unsupervised learning finds structure in unlabeled data; reinforcement learning learns actions through rewards and penalties. Give one real use case for each.

Common pitfall / senior-level tip: Avoid describing RL as simply "unlabeled learning." The agent interacts with an environment and optimizes cumulative reward.

2. What is the difference between classification and regression?

What a strong answer should cover: Classification predicts discrete classes; regression predicts continuous values. Mention examples, loss functions, and evaluation metrics.

Common pitfall / senior-level tip: Do not use accuracy for regression. Think MAE, MSE, RMSE, and R-squared.

3. Explain bias and variance.

What a strong answer should cover: Bias is error from overly simple assumptions and tends to cause underfitting. Variance is sensitivity to training-data noise and tends to cause overfitting.

Common pitfall / senior-level tip: Connect the concept to the bias-variance tradeoff and model complexity.

4. What is overfitting and how do you prevent it?

What a strong answer should cover: Use cross-validation, regularization, early stopping, dropout for neural networks, data augmentation, simpler models, feature selection, and more representative data.

Common pitfall / senior-level tip: Mention the generalization gap between training and validation performance.

5. What is underfitting and how do you fix it?

What a strong answer should cover: Use a more expressive model, better features, less aggressive regularization, more training, or improve data quality.

Common pitfall / senior-level tip: Do not assume adding complexity is always the answer; poor features or noisy labels may be the real issue.

6. Explain train, validation, and test sets.

What a strong answer should cover: Train fits parameters, validation selects/tunes the approach, and test provides the final unbiased estimate. Keep the test set untouched until the end.

Common pitfall / senior-level tip: Explain data leakage when preprocessing or tuning uses test data.

7. What is cross-validation and when would you use it?

What a strong answer should cover: K-fold cross-validation repeatedly trains on k-1 folds and validates on the remaining fold. It is useful when data is limited and a stable performance estimate is needed.

Common pitfall / senior-level tip: For time-series data, use time-aware splits rather than random K-fold.

8. What is data leakage?

What a strong answer should cover: Leakage occurs when information unavailable at prediction time influences training or validation. Prevent it by fitting preprocessing only on training data and designing features from point-in-time information.

Common pitfall / senior-level tip: Give a concrete example such as using a future transaction outcome as a feature.

9. Explain feature scaling. When is it necessary?

What a strong answer should cover: Standardization or normalization puts features on comparable scales. It is important for distance-based models, gradient-based optimization, SVMs, and many neural networks; it is usually less critical for tree models.

Common pitfall / senior-level tip: Do not blindly scale every feature pipeline.

10. What is the difference between normalization and standardization?

What a strong answer should cover: Normalization commonly maps values to a fixed range such as [0,1]. Standardization transforms using mean and standard deviation: $z=(x-\mu)/\sigma$.

Common pitfall / senior-level tip: Be clear that terminology can vary across teams and libraries.

Section 2 - Core Algorithms and Metrics

11. Explain linear regression and its assumptions.

What a strong answer should cover: Linear regression models a linear relationship between features and target. Common assumptions include linearity, independent errors, constant error variance, and limited multicollinearity.

Common pitfall / senior-level tip: For prediction, focus on whether assumptions affect the intended use, not only textbook definitions.

12. What is logistic regression?

What a strong answer should cover: A linear classifier that maps a weighted feature sum through the sigmoid function to estimate class probability. It is often interpreted through coefficients and is a strong baseline.

Common pitfall / senior-level tip: It is a classification algorithm despite the name "regression."

13. Compare decision trees, random forests, and gradient boosting.

What a strong answer should cover: Trees are easy to interpret but can overfit. Random forests use bagging and feature randomness to reduce variance. Gradient boosting adds weak learners sequentially to reduce residual error and often delivers strong tabular performance.

Common pitfall / senior-level tip: Mention tuning, interpretability, training cost, and robustness depending on the situation.

14. Explain SVM at a high level.

What a strong answer should cover: SVM finds a decision boundary with maximum margin and can use kernels for non-linear separation. C controls the margin/error tradeoff and gamma affects the influence of individual examples in RBF kernels.

Common pitfall / senior-level tip: Know that SVMs can be expensive on very large datasets.

15. What is K-means clustering and what are its limitations?

What a strong answer should cover: K-means assigns points to k centroids and iteratively minimizes within-cluster squared distance. Limitations include choosing k, sensitivity to initialization/outliers, and assumptions about roughly spherical clusters.

Common pitfall / senior-level tip: Mention K-means++ initialization and scaling features.

16. What are precision, recall, and F1-score?

What a strong answer should cover: Precision=TP/(TP+FP). Recall=TP/(TP+FN). F1 is the harmonic mean of precision and recall. Prioritize precision when false positives are costly and recall when false negatives are costly.

Common pitfall / senior-level tip: Never confuse FP with FN in the formulas.

17. What is ROC-AUC vs PR-AUC?

What a strong answer should cover: ROC-AUC summarizes ranking quality across thresholds using TPR vs FPR. PR-AUC focuses on precision-recall tradeoffs and is often more informative for highly imbalanced positive classes.

Common pitfall / senior-level tip: For rare-event problems, explicitly discuss PR-AUC.

18. How do you handle class imbalance?

What a strong answer should cover: Use class weights/cost-sensitive learning, oversampling such as SMOTE when appropriate, undersampling, threshold tuning, and imbalance-aware metrics. Use stratified splits for evaluation.

Common pitfall / senior-level tip: Stratified K-fold preserves class proportions; it does not itself solve the imbalance.

19. What is regularization? Compare L1 and L2.

What a strong answer should cover: Regularization penalizes model complexity. L1 encourages sparse coefficients and can perform feature selection; L2 shrinks coefficients smoothly and is often more stable with correlated features.

Common pitfall / senior-level tip: Know that elastic net combines L1 and L2.

20. How do you choose the right evaluation metric?

What a strong answer should cover: Start from business cost and error types, then select metrics aligned to the objective. Consider threshold behavior, calibration, class imbalance, ranking vs classification, and operational constraints.

Common pitfall / senior-level tip: A senior answer connects the metric to a decision or business outcome.

Section 3 - Deep Learning

21. What is a neural network and what happens during forward and backward propagation?

What a strong answer should cover: Forward propagation computes predictions layer by layer. Backpropagation computes gradients of the loss with respect to parameters using the chain rule; an optimizer updates weights.

Common pitfall / senior-level tip: Mention learning rate and loss function when explaining training.

22. Compare common activation functions.

What a strong answer should cover: ReLU is common in hidden layers due to simplicity; sigmoid is useful for binary output probabilities; tanh is zero-centered but can saturate; softmax is common for multi-class probability outputs.

Common pitfall / senior-level tip: Know the dying ReLU issue and common alternatives such as Leaky ReLU/GELU.

23. What is dropout and why does it help?

What a strong answer should cover: Dropout randomly removes activations during training, discouraging co-adaptation and acting as regularization. It is disabled or scaled appropriately during inference.

Common pitfall / senior-level tip: Do not describe dropout as permanently deleting neurons.

24. What is batch normalization?

What a strong answer should cover: It normalizes intermediate activations using batch statistics during training and running statistics at inference. It can stabilize and accelerate training and adds some regularization.

Common pitfall / senior-level tip: Know that placement and interaction with network architecture matter.

25. What is the vanishing/exploding gradient problem?

What a strong answer should cover: Gradients can become extremely small or large through deep chains of derivatives, making training unstable or very slow. Mitigations include better initialization, normalization, residual connections, suitable activations, gradient clipping, and gated architectures.

Common pitfall / senior-level tip: Tie the issue to depth and activation behavior.

26. Compare CNNs, RNNs/LSTMs, and Transformers.

What a strong answer should cover: CNNs excel at spatial/local patterns; RNNs model sequences recursively, with LSTMs/GRUs addressing long dependencies; Transformers use attention to model relationships in parallel and dominate many modern language tasks.

Common pitfall / senior-level tip: Be ready to explain why attention improves parallelism and long-range dependency handling.

27. What is attention in a Transformer?

What a strong answer should cover: Attention computes relationships between tokens using queries, keys, and values. Self-attention lets each token weigh information from other tokens to build contextual representations.

Common pitfall / senior-level tip: Know the basic scaled dot-product attention formula conceptually.

28. What is transfer learning and when would you use it?

What a strong answer should cover: Start from a pretrained model and adapt it to a target task. It is useful when labeled target data is limited or pretrained representations are strong.

Common pitfall / senior-level tip: Discuss freezing layers, fine-tuning, learning rates, and domain shift.

29. What is fine-tuning vs prompting in an LLM?

What a strong answer should cover: Prompting changes task instructions at inference time without changing model weights. Fine-tuning updates model parameters on task/domain data. Choose based on data, cost, stability, and control requirements.

Common pitfall / senior-level tip: Parameter-efficient tuning such as LoRA is a valuable senior-level follow-up.

30. How do you prevent overfitting in deep learning?

What a strong answer should cover: Use dropout, weight decay, early stopping, data augmentation, transfer learning, smaller architectures, validation monitoring, and good dataset splits.

Common pitfall / senior-level tip: Mention the training/validation loss curves and generalization behavior.

Section 4 - MLOps and Production ML

31. Walk me through an end-to-end ML project lifecycle.

What a strong answer should cover: Define business problem -> collect/validate data -> split -> explore/features -> train -> evaluate -> package -> deploy -> monitor -> retrain. Include reproducibility and governance.

Common pitfall / senior-level tip: The strongest answers explain feedback loops after deployment.

32. What is model drift? Differentiate data, concept, and prediction drift.

What a strong answer should cover: Data drift changes feature distributions. Concept drift changes the relationship between inputs and target. Prediction drift changes model output distribution. Detection should be tied to business impact and labels when available.

Common pitfall / senior-level tip: Mention that drift does not automatically mean model quality has degraded.

33. What should you monitor after deploying a model?

What a strong answer should cover: Input quality, feature drift, missingness, prediction distribution, latency, throughput, errors, resource use, and business KPIs. When labels arrive, monitor performance and calibration too.

Common pitfall / senior-level tip: Monitoring needs actionable thresholds and alert ownership.

34. How would you deploy an ML model as an API?

What a strong answer should cover: Package the model with preprocessing, expose an endpoint, validate inputs, containerize, deploy behind a service, add logging/metrics, version artifacts, and support rollback.

Common pitfall / senior-level tip: Discuss cold start, concurrency, model loading, and backward compatibility.

35. What is CI/CD for ML systems?

What a strong answer should cover: Automate tests, data/schema checks, model validation, packaging, deployment, and rollback. ML adds data and model versioning beyond normal application CI/CD.

Common pitfall / senior-level tip: Mention reproducible training and approval gates.

36. What is MLflow or a similar experiment-tracking platform used for?

What a strong answer should cover: Track parameters, metrics, artifacts, model versions, and experiments so results are reproducible and comparable. It can also support model registry and lifecycle management.

Common pitfall / senior-level tip: Distinguish experiment tracking from feature stores and orchestration tools.

37. Why is feature engineering important? What is a feature store?

What a strong answer should cover: Good features improve signal and model efficiency. A feature store centralizes reusable, versioned features and supports consistent offline training and online serving.

Common pitfall / senior-level tip: Point-in-time correctness is a key senior-level concept.

38. How do you handle a model that performs well offline but poorly in production?

What a strong answer should cover: Check train/serve skew, data leakage, distribution shift, feature freshness, preprocessing mismatch, thresholding, latency constraints, and metric mismatch. Reproduce production examples and compare distributions.

Common pitfall / senior-level tip: This is a classic production debugging question: explain your investigation order.

39. How do you make an ML pipeline reproducible?

What a strong answer should cover: Version code, data snapshots or references, features, environments, model artifacts, random seeds, configuration, and dependencies. Record lineage and experiment metadata.

Common pitfall / senior-level tip: Reproducibility is broader than setting `random_state`.

40. Batch inference vs online inference: when would you choose each?

What a strong answer should cover: Batch is suitable for periodic large-scale predictions where low latency is unnecessary. Online inference is for real-time decisions and user-facing experiences.

Common pitfall / senior-level tip: Discuss cost, freshness, latency, throughput, and operational complexity.

Section 5 - GenAI, RAG, LLMs, and System Design

41. What is RAG and why is it useful?

What a strong answer should cover: Retrieval-Augmented Generation retrieves relevant external context and supplies it to an LLM before generation. It helps ground responses in domain-specific or changing information without retraining the base model.

Common pitfall / senior-level tip: Explain the pipeline: ingest -> chunk -> embed -> retrieve -> rerank -> prompt -> generate -> evaluate.

42. How would you design a production RAG system?

What a strong answer should cover: Cover ingestion, parsing, chunking, metadata, embeddings, vector/keyword retrieval, hybrid search, reranking, prompt construction, generation, citations, caching, evaluation, access control, and monitoring.

Common pitfall / senior-level tip: A senior answer balances relevance, latency, cost, and security.

43. How do embeddings work conceptually?

What a strong answer should cover: An embedding maps text/items into vectors where semantic similarity is represented geometrically. Similar meanings tend to be closer under a chosen similarity metric.

Common pitfall / senior-level tip: Mention cosine similarity, dimensionality, and domain-specific embedding quality.

44. How do you reduce hallucinations in an LLM application?

What a strong answer should cover: Use high-quality retrieval, reranking, grounded prompts, citations, constrained output formats, tool use, confidence/abstention strategies, and evaluation. Instruct the model to say when evidence is insufficient.

Common pitfall / senior-level tip: Avoid promising that prompting alone eliminates hallucinations.

45. What is chunking and how do you choose chunk size?

What a strong answer should cover: Chunking breaks source content into retrieval units. Choose size and overlap based on document structure, semantic boundaries, model context, retrieval precision, and downstream task.

Common pitfall / senior-level tip: Test chunking empirically; there is no universally correct size.

46. How would you evaluate a RAG system?

What a strong answer should cover: Measure retrieval quality (recall@k, precision@k, MRR), answer correctness/groundedness, citation quality, latency, cost, and failure rates. Use curated test sets and human or model-assisted evaluation with safeguards.

Common pitfall / senior-level tip: Separate retrieval failures from generation failures.

47. What are common LLM application security risks?

What a strong answer should cover: Prompt injection, data exfiltration, insecure tool use, over-privileged agents, sensitive-data leakage, malicious documents, and weak output validation. Mitigate with least privilege, isolation, allowlists, filtering, and audit logging.

Common pitfall / senior-level tip: Do not treat the LLM as a trusted execution environment.

48. What is an AI agent and how is it different from a chatbot?

What a strong answer should cover: A chatbot mainly generates responses. An agent can plan, select tools, execute actions, observe results, and iterate toward a goal. Tool permissions and state management are central design concerns.

Common pitfall / senior-level tip: Explain bounded autonomy and guardrails.

49. Design an AI assistant that generates a milestone/release report from engineering data.

What a strong answer should cover: Use source connectors/APIs for milestones, issues, MRs, commits, pipelines, and ownership. Normalize data, retrieve relevant records, summarize by team/feature/bug, calculate release readiness indicators, cite source records, and expose via an API/agent interface. Add access control, caching, retries, and evaluation.

Common pitfall / senior-level tip: Expect follow-ups on MCP/tool design, data freshness, permissions, and hallucination prevention.

50. Describe a challenging ML/AI project you owned end-to-end. What would you do differently?

What a strong answer should cover: Use STAR: situation, task, actions, measurable result, and lessons learned. Explain architecture, tradeoffs, your individual contribution, production impact, failures, and what you would improve.

Common pitfall / senior-level tip: For a five-year role, interviewers look for ownership, technical depth, debugging ability, and business impact - not just a list of technologies.

Rapid Review - 25 Must-Know Questions

1. Supervised vs unsupervised vs reinforcement learning
2. Bias vs variance
3. Overfitting prevention
4. Train/validation/test split
5. Data leakage
6. Precision vs recall
7. ROC-AUC vs PR-AUC
8. Class imbalance
9. L1 vs L2 regularization
10. Decision tree vs random forest vs boosting
11. Neural network forward/backpropagation
12. ReLU/sigmoid/softmax
13. Dropout
14. Vanishing/exploding gradients
15. CNN vs RNN vs Transformer
16. Transfer learning and fine-tuning
17. ML project lifecycle
18. Data/model/concept drift
19. Production monitoring
20. Offline-good / production-bad debugging
21. RAG architecture
22. Embeddings and vector search
23. Hallucination reduction
24. LLM security and prompt injection
25. AI agent design and tool permissions

Mock interview scoring rubric

Score	Meaning
90-100%	Excellent: accurate, structured, senior-level tradeoffs and production examples.
75-89%	Strong: mostly correct with minor gaps; good practical understanding.
60-74%	Developing: core concept is present but explanation or depth is incomplete.
40-59%	Needs improvement: partial understanding, formula/example errors, or missing key points.
Below 40%	Re-study the concept and practice explaining it from first principles.